



Security Review For MO

A large black circle containing the white text "MO" in a stylized, bold, sans-serif font.

Collaborative Audit Prepared For:
Lead Security Expert(s):

MO
Oxeix
defsec
ParthMandale
pkqs90
SBSecurity
April 20 - April 26, 2026

Date Audited:

Introduction

PYUSDx is a managed ERC-20 stablecoin featuring a configurable interest-bearing mechanism based on a continuous compounding index. The system includes extension contracts, which are ERC-20 tokens that allow users to wrap and unwrap PYUSDx to create white-label stablecoins. Additionally, a SwapFacility enables users to swap tokens between PYUSDx, approved extensions, and whitelisted assets. Finally, a cross-chain Portal facilitates token transfers across different networks using bridge adapters.

Scope

Repository: [m0-foundation/PYUSDx](https://github.com/m0-foundation/PYUSDx)

Audited Commit: [f56dccf2db25c0be29775e5e871d87da8c0b5996](https://github.com/m0-foundation/PYUSDx/commit/f56dccf2db25c0be29775e5e871d87da8c0b5996)

Final Commit: [b33935a7c1feef9c9756e6b3e51a1efc2859b2c0](https://github.com/m0-foundation/PYUSDx/commit/b33935a7c1feef9c9756e6b3e51a1efc2859b2c0)

Files:

- [src/abstract/interfaces/IRateLimiter.sol](#)
- [src/abstract/RateLimiter.sol](#)
- [src/core/IIssuerGateway.sol](#)
- [src/core/IssuerGateway.sol](#)
- [src/IPYUSDx.sol](#)
- [src/platform/ExtensionBeaconProxy.sol](#)
- [src/platform/ExtensionBeacon.sol](#)
- [src/platform/ExtensionFactory.sol](#)
- [src/platform/Extension.sol](#)
- [src/platform/interfaces/IExtensionBeacon.sol](#)
- [src/platform/interfaces/IExtensionFactory.sol](#)
- [src/platform/interfaces/IExtension.sol](#)
- [src/platform/projects/interfaces/IMultiMint.sol](#)
- [src/platform/projects/interfaces/IYieldToOne.sol](#)
- [src/platform/projects/MultiMint.sol](#)
- [src/platform/projects/YieldToOne.sol](#)
- [src/portal/bridgeAdapters/BridgeAdapter.sol](#)
- [src/portal/bridgeAdapters/layerZero/interfaces/ILayerZeroBridgeAdapter.sol](#)
- [src/portal/bridgeAdapters/layerZero/interfaces/ILayerZeroEndpointV2.sol](#)

- src/portal/bridgeAdapters/layerZero/interfaces/ILayerZeroReceiver.sol
- src/portal/bridgeAdapters/layerZero/LayerZeroBridgeAdapter.sol
- src/portal/bridgeAdapters/layerZero/libraries/ExecutorOptions.sol
- src/portal/bridgeAdapters/layerZero/libraries/OptionsBuilder.sol
- src/portal/interfaces/IBridgeAdapter.sol
- src/portal/interfaces/IPortal.sol
- src/portal/libraries/PayloadEncoder.sol
- src/portal/Portal.sol
- src/portal/ReentrancyLock.sol
- src/PYUSDx.sol
- src/swap/interfaces/IMsgSender.sol
- src/swap/interfaces/IReentrancyLock.sol
- src/swap/interfaces/ISwapFacility.sol
- src/swap/ReentrancyLock.sol
- src/swap/SwapFacility.sol

Final Commit Hash

b33935a7c1feef9c9756e6b3e51a1efc2859b2c0

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	17

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: IssuerGateway mint delay and TTL are mutable retroactively [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/35>

Summary

IssuerGateway.proposeMint() stores only createdAt on the proposal and recomputes activeAt = createdAt + \$.mintDelay at execution time during minting. Because _setMintDelay accepts an arbitrary value and no timelock, DEFAULT_ADMIN_ROLE can shorten or increase the mint delay after a proposal has been created. Same thing also applies to expiration - if the proposal has expired, it technically can be revived by admin actions.

Vulnerability Detail

Currently, mint proposals only store:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/core/IssuerGateway.sol#L114-119>

```
$.mintProposals[mintId] = MintProposal({
    createdAt: createdAt,
    minter: msg.sender,
    recipient: recipient,
    amount: amount
});
```

They don't store the activeAt or expiresAt values that were emitted at proposal time. Therefore, mint() and cancelMint() functions recompute timing from the current global values:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/core/IssuerGateway.sol#L160-162>

```
unchecked {
    activeAt = proposal.createdAt + $.mintDelay;
}
```

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/core/IssuerGateway.sol#L166-168>

```
unchecked {
    expiresAt = activeAt + $.mintTTL;
}
```

This means the admin can change the rules for already-created proposals.

Impact

The timelock can be bypassed and expired proposals can also be revived - it doesn't have to be necessarily a malicious action, changing parameters is a normal admin action that can happen at any moment.

Tool Used

Manual Review

Recommendation

Consider introducing snapshot timing per proposal:

```
struct MintProposal {
    uint40 createdAt;
+   uint40 activeAt;
+   uint40 expiresAt;
    address minter;
    address recipient;
    uint256 amount;
}
```

Discussion

rodiontr

closed as expected behavior per spec

PierrickGT

Fixed in the following PR: <https://github.com/m0-foundation/PYUSDx/pull/52>

Issue L-2: MultiMint.replaceAsset() bypasses extension freeze checks [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/36>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

MultiMint own Freezable list for either the original caller or the recipient in the replaceAsset() function.

Vulnerability Detail

MultiMint.wrap() passes the original user through the SwapFacility and eventually checks freeze status:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/platform/projects/MultiMint.sol#L142-144>

```
_wrapAsset(asset, ISwapFacility(msg.sender).msgSender(), recipient, amount);
```

Then _beforeWrap() calls:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/platform/projects/MultiMint.sol#L221>

```
super._beforeWrap(account, recipient, amount);
```

which checks pause/freeze. But replaceAsset() does not pass the original caller:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/platform/projects/MultiMint.sol#L147-149>

```
function replaceAsset(address asset, address recipient, uint256 amount) external  
↪ onlySwapFacility {  
    _replaceAsset(asset, recipient, amount);  
}
```

And _replaceAsset() only checks pause, invalid asset, zero recipient, zero amount, and backing:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/platform/projects/MultiMint.sol#L280-284>

```

_requireNotPaused();

_revertIfInvalidAsset(asset);
_revertIfZeroAccount(recipient);
_revertIfZeroAmount(amount);

```

It does not check whether the original caller is frozen in the `MultiMint` extension. It also does not check whether the recipient is frozen in that extension.

Impact

If extension-level freezing is meant to block sanctioned/compromised users from interacting with that extension's reserves (as `replaceAsset()` bypasses it. The only place where the behavior is documented is the following test file:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDx/test/unit/MultiMintUnit.t.sol#L567-568>

```

// Succeeds because replaceAsset does not check if the caller or recipient are
↪ frozen.
    swapFacility.replaceAsset(address(extension), address(usdc), 50e6, bob);

```

However, that still affects `MultiMint`'s freeze as an isolation mechanism and frozen users can still rotate value in/out of `MultiMint`'s asset reserves.

Tool Used

Manual Review

Recommendation

Change the interface to pass the original account:

```

function replaceAsset(
    address asset,
+   address account,
    address recipient,
    uint256 amount
) external onlySwapFacility;

```

Then call a hook similar to:

```

+ function _beforeReplaceAsset(address account, address recipient, uint256 amount)
↪ internal view {
+   _requireNotPaused();

```



```
+ FreezableStorageStruct storage $ = _getFreezableStorageLocation();  
+ _revertIfFrozen($, account);  
+ _revertIfFrozen($, recipient);  
+ }
```

Discussion

PierrickGT

Acknowledged. This is by design. The caller address should be frozen at the PYUSDx and/or secondary backing asset level if this address is not supposed to interact with these tokens. Same goes for the recipient. Additionally, the Asset Cap Manager can now set a whitelist and only allow some addresses to call `replaceAsset`.

<https://github.com/m0-foundation/PYUSDx/pull/62>

Issue L-3: LZ delegate persists after operator revocation enabling in transit message loss [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/37>

Summary

LayerZeroBridgeAdapter._revokeRole() overrides the standard access-control revocation to automatically clear the LayerZero delegate when an OPERATOR_ROLE holder is revoked. However, the clearing logic contains a conditional that only clears the delegate if the current delegate address matches the account being revoked.

Vulnerability Detail

An operator can call setDelegate(thirdPartyAddress) before being revoked. When revocation occurs, delegates(address(this)) == account evaluates to false (because delegates(this) = thirdPartyAddress != operator), so the delegate is not cleared. The third-party address retains the LayerZero delegate role indefinitely, with authority to call skip() on the LZ endpoint to permanently discard inbound messages.

The LayerZero skip() function permanently discards a queued message at a specific nonce. If the source chain has already burned PYUSDx tokens and transmitted the bridge message, and the malicious delegate calls skip() for that message on the destination endpoint, no mint occurs on the destination chain, the tokens are permanently lost.

```
// LayerZeroBridgeAdapter.sol:91-101
function _revokeRole(bytes32 role, address account) internal override returns
↳ (bool) {
    bool revoked = super._revokeRole(role, account);
    if (revoked && role == OPERATOR_ROLE) {
        if (ILayerZeroEndpointV2(endpoint).delegates(address(this)) == account) {
            ILayerZeroEndpointV2(endpoint).setDelegate(address(0));
        }
    }
    return revoked;
}
```

Impact

A malicious or compromised operator can pre-configure a third-party delegate before revocation, leaving the adapter in a state where: (1) the operator loses their protocol role, and (2) an external address retains LayerZero configuration authority including the

ability to permanently discard in-flight messages. Any PYUSDx burned on the source chain in the window between the delegate's installation and the admin's recovery action is at risk of being permanently lost.

Code Snippet

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDx/src/portal/bridgeAdapters/layerZero/LayerZeroBridgeAdapter.sol#L96>

Tool Used

Manual Review

Recommendation

Remove the conditional check and unconditionally clear the delegate whenever any OPERATOR_ROLE is revoked:

```
if (revoked && role == OPERATOR_ROLE) {  
-     if (ILayerZeroEndpointV2(endpoint).delegates(address(this)) == account) {  
-         ILayerZeroEndpointV2(endpoint).setDelegate(address(0));  
-     }  
+     // Always clear the delegate when any operator is revoked,  
+     // regardless of current delegate value.  
+     ILayerZeroEndpointV2(endpoint).setDelegate(address(0));  
}
```

Issue L-4: Malicious bridge adapter combined with unconfigured rate limit enables supply inflation [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/38>

Summary

The Portal's `receiveMessage()` function mints arbitrary PYUSDx when called by any registered bridge adapter, and the `RateLimiter` operates in fail-open mode for unconfigured or deleted rate limit buckets. A compromised `OPERATOR_ROLE` can register a malicious bridge adapter, and a compromised `RATE_LIMIT_MANAGER_ROLE` can delete the Portal's rate limit bucket, enabling unlimited PYUSDx supply inflation with no on-chain cap.

Vulnerability Detail

The attack chain involves two semi-trusted roles acting in concert:

1. `RATE_LIMIT_MANAGER` calls `setRateLimit(portal, 0, 0, false)`, which executes `delete $.issuerBuckets[portal]` and resets `lastRefillTime` to zero.
2. `OPERATOR` calls `setSupportedBridgeAdapter(chainId, maliciousAdapter, true)`, registering an attacker-controlled contract as a valid bridge adapter.
3. The malicious adapter calls `portal.receiveMessage(sourceChainId, craftedPayload)` with an arbitrary mint amount and recipient. It passes the bridge adapter check because it was registered in step 2.
4. Portal calls `PYUSDx.mint(recipient, amount)`. Inside `_mint`, `_enforceRateLimit(msg.sender, amount)` is invoked with the Portal's address as the issuer. Because the bucket was deleted in step 1, `lastRefillTime == 0`, and the rate limiter returns immediately without any capacity check (fail-open).
5. Steps 3-4 can be repeated indefinitely with different message IDs, inflating PYUSDx supply without limit.

The fail-open behavior is the core code-level issue. Even if the rate limit was properly configured at deployment, `RATE_LIMIT_MANAGER_ROLE` can delete it at any time, reverting the issuer to "unlimited capacity" status.

Impact

Unlimited PYUSDx minting with no corresponding burn on any source chain. An attacker who compromises or controls the `OPERATOR` and `RATE_LIMIT_MANAGER` roles (both

semi-trusted, not the highest-privilege admin) can mint arbitrary quantities of PYUSDx, leading to total protocol insolvency, all minted tokens have no backing, devaluing every legitimate holder's tokens to zero.

Code Snippet

RateLimiter.sol:128-132 – fail-open rate limit check:

```
function _enforceRateLimit(address issuer, uint256 amount) internal {
    Bucket storage bucket = _getRateLimiterStorage().issuerBuckets[issuer];

    // NOTE: Unconfigured issuers have unlimited capacity
    if (bucket.lastRefillTime == 0) return; // returns without checking amount
```

RateLimiter.sol:56-72 – rate limit deletion:

```
function setRateLimit(address issuer, uint128 capacity, uint128 refillPerSecond,
    ↪ bool enabled)
    external onlyRole(RATE_LIMIT_MANAGER_ROLE)
{
    if (!enabled) {
        if (capacity != 0 || refillPerSecond != 0) revert
            ↪ InvalidRateLimitRemoval(capacity, refillPerSecond);
        delete $.issuerBuckets[issuer]; // resets lastRefillTime to 0
        return;
    }
}
```

Portal.sol:163-205 – any registered adapter triggers minting:

```
function receiveMessage(uint32 sourceChainId, bytes calldata payload) external
    ↪ whenReceiveNotPaused whenNotLocked {
    _revertIfUnsupportedBridgeAdapter(sourceChainId, msg.sender);
    // ... decodes payload with arbitrary amount and recipient ...
    if (destinationToken == pyusdx) {
        IPYUSDx(pyusdx).mint(recipient, amount);
    }
}
```

PYUSDx.sol:602-616 – _mint calls _enforceRateLimit with msg.sender (Portal) as issuer:

```
function _mint(address account, uint256 amount) internal virtual {
    _revertIfZeroAccount(account);
    _revertIfFrozen(account);
    _revertIfZeroAmount(amount);

    _enforceRateLimit(msg.sender, amount);

    PYUSDxStorage storage $ = _getPYUSDxStorage();
```

```
    $.totalSupply += amount;  
    // ...  
}
```

Tool Used

Manual Review

Recommendation

Change the rate limiter's default behavior from fail-open to fail-closed for unconfigured issuers:

```
- // NOTE: Unconfigured issuers have unlimited capacity  
- if (bucket.lastRefillTime == 0) return;  
+ // Unconfigured issuers are BLOCKED (fail-closed)  
+ if (bucket.lastRefillTime == 0) revert RateLimitNotConfigured(issuer);
```

Discussion

PierrickGT

The proposed solution does not really address the main issue since once the `RATE_LIMIT_MANAGER_ROLE` is compromised, the attacker would be able to set a high rate limit and mint a large amount of unbacked PYUSDx anyway. That being said, making the rate limit configuration explicit rather than implicit is a good practice, even if it increases configuration overhead. For this reason, we have implemented the recommendation.

Issue L-5: Changing earnerManager redirects all unclaimed fees to the new manager [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/39>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

PYUSDx does not snapshot a fee recipient per account - instead, it pays accrued fees to whoever is the current earnerManager at claim time. Rotating earnerManager therefore retroactively reassigns all unclaimed, already accrued fees to the new manager.

Vulnerability Detail

Fees are not accrued to a stored fee recipient at the time they are earned. They are calculated at claim time and sent to the current earnerManager:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDx/src/PYUSDx.sol#L474-478>

```
address feeRecipient = $.earnerManager;

    emit FeeClaimed(account, feeRecipient, fee);

    _transfer(account, feeRecipient, fee);
```

If earnerManager changes, all previously accrued but unclaimed fees will be paid to the new manager.

Impact

The new manager can receive fees for periods before it was manager. While the probability of that is quite low (to cause a substantial loss), the loss of funds can still happen.

Tool Used

Manual Review

Recommendation

Before changing `earnerManager`, make sure the previous manager gets the fees for the previous period.

Discussion

PierrickGT

Acknowledged. Since the Earner Manager is not meant to be changed often, we will claim for the accounts it manages before changing of Earner Manager.

Issue L-6: Portal ignores destinationChainId and destinationPeer in cross chain payload [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/40>

Summary

The payload encoder writes destinationChainId (4 bytes) and destinationPeer (32 bytes) in the first 36 bytes of every cross-chain message, but the decoder skips those fields and Portal's receiveMessage never validates them. Routing correctness currently depends entirely on each bridge adapter's internal checks.

Vulnerability Detail

The encoder packs six fields including destinationChainId and destinationPeer, but the decoder skips the first 36 bytes without reading them:

```
// PayloadEncoder.sol:94-102
uint256 offset = DESTINATION_CHAIN_ID_LENGTH + DESTINATION_PEER_LENGTH;
bytes32 destinationTokenBytes32;
bytes32 recipientBytes32;

(messageId, offset) = payload.asBytes32Unchecked(offset);
(amount, offset) = payload.asUint128Unchecked(offset);
```

And Portal's receive path never compares these fields to currentChainId() or address(this):

```
// Portal.sol:163-167
function receiveMessage(uint32 sourceChainId, bytes calldata payload) external
↪ whenReceiveNotPaused whenNotLocked {
    _revertIfUnsupportedBridgeAdapter(sourceChainId, msg.sender);

    (bytes32 messageId, uint256 amount, address destinationToken, bytes32 sender,
    ↪ address recipient) = payload
        .decodeTokenTransfer();
```

Impact

Any future adapter addition is a trust-assumption upgrade with no Portal-layer safety net.

Code Snippet

```
// portal/libraries/PayloadEncoder.sol:85-108
function decodeTokenTransfer(
    bytes memory payload
)
    internal
    pure
    returns (bytes32 messageId, uint256 amount, address destinationToken, bytes32
        ↪ sender, address recipient)
{
    if (payload.length != PAYLOAD_LENGTH) revert
        ↪ InvalidPayloadLength(payload.length);

    uint256 offset = DESTINATION_CHAIN_ID_LENGTH + DESTINATION_PEER_LENGTH; // <--
        ↪ skip 36 bytes, never validated
    bytes32 destinationTokenBytes32;
    bytes32 recipientBytes32;

    (messageId, offset) = payload.asBytes32Unchecked(offset);
    (amount, offset) = payload.asUint128Unchecked(offset);
    (destinationTokenBytes32, offset) = payload.asBytes32Unchecked(offset);
    (sender, offset) = payload.asBytes32Unchecked(offset);
    (recipientBytes32, offset) = payload.asBytes32Unchecked(offset);
```

Tool Used

Manual Review

Recommendation

Return the two fields from `decodeTokenTransfer` and enforce them in `Portal.receiveMessage`.

Issue L-7: Expired IssuerGateway proposals cannot be cleaned up [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/41>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Once a `MintProposal` expires, it becomes permanently uncancellable and unexecutable, but its storage slot is never freed. There is no function to remove expired proposals from `$.mintProposals`.

Vulnerability Detail

After a proposal becomes active, the proposer cannot cancel it:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/core/IssuerGateway.sol#L136-140>

```
unchecked {  
    activeAt = proposal.createdAt + $.mintDelay;  
}  
  
if (block.timestamp >= activeAt) revert ActiveMintProposal(activeAt);
```

But the expired proposal remains in storage. There is no cleanup function.

Impact

Storage bloat.

Tool Used

Manual Review

Recommendation

Add a function to delete the expired proposals allow the proposer/admin to cancel expired proposals.

Discussion

PierrickGT

Acknowledged. We won't implement this recommendation because it expands the state machine/code surface, and keeping expired proposals in storage isn't an issue. It can also be useful to keep expired proposals for audit-ability in the future.

Issue L-8: totalSupply does not include unclaimed yield [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/42>

Summary

The PYUSDx specification states:

```
totalSupply equals the sum of all account balances plus all unclaimed yield.
```

However, the implementation does not satisfy this. `totalSupply` reflects only minted tokens and pending earner yield is a virtual liability that is not included in `totalSupply` until `_claimFor` runs.

Vulnerability Detail

`totalSupply()` returns only the stored supply:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDx/src/PYUSDx.sol#L212-214>

```
function totalSupply() public view returns (uint256) {  
    return _getPYUSDxStorage().totalSupply;  
}
```

Unclaimed yield is not added to `$.totalSupply` until claim time:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDx/src/PYUSDx.sol#L452>

```
$.totalSupply += yieldWithFee;
```

So before claim:

```
totalSupply = sum of stored balances
```

not:

```
sum of stored balances + unclaimed yield
```

Impact

This is either: a spec bug, because the intended token model is `yield is minted only when claimed`; or a code bug, because the implementation fails the stated invariant.

Tool Used

Manual Review

Recommendation

Consider updating the spec, or add a separate view/helper if the protocol wants to expose fully accrued supply.

Issue L-9: User's earningPrincipal may silently overflow uint112 [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/43>

Vulnerability Detail

PYUSDx stores spendable balance as uint256, but stores the earning base as earningPrincipal in uint112. When an earning account receives tokens, `_addEarningAmount()` converts the incoming amount to principal and adds it to earningPrincipal.

```
function _addEarningAmount(PYUSDxStorage storage $, address account, uint256
↪ amount) internal {
    uint112 principal = _getPrincipalAmountRoundedDown(amount,
    ↪ _updateIndexOf(account));

    unchecked {
        $.accounts[account].balance += amount;
        $.accounts[account].earningPrincipal += principal;
    }
}
```

Because the addition is unchecked, an earning account near `type(uint112).max` can receive more tokens and wrap `earningPrincipal` to a much smaller value instead of reverting. The transfer still succeeds and the recipient keeps the higher balance, but future yield is computed from the wrapped `earningPrincipal`, so accrual becomes incorrect or zero.

Note the balance can also overflow but since that is in u256 type, it is realistically impossible to overflow this. However, for `earningPrincipal`, since it is u112 type, it is much more likely.

Proof of Concept

The following test models a near-cap earning account at 1x index, sends it 100 more units, and shows that `earningPrincipal` wraps to 49 while balance increases normally. After one year, `accruedYieldOf()` is 0 because the yield base has been corrupted.

```
function test_poc_earningPrincipalWrapsOnInboundTransfer() public {
    vm.startPrank(earnerManager);
    pyusdx.setAccountInfo(alice, 500, 0, address(0));
    pyusdx.setAccountInfo(bob, 500, 0, address(0));
    vm.stopPrank();

    pyusdx.setAccountLastIndex(alice, PRECISION);
```

```
pyusdx.setAccountLastIndex(bob, PRECISION);

pyusdx.setBalance(bob, uint256(type(uint112).max) - 50);
pyusdx.setEarningPrincipal(bob, type(uint112).max - 50);
pyusdx.setTotalSupply(uint256(type(uint112).max) - 50);

issuerGateway.mint(alice, 100);

vm.prank(alice);
pyusdx.transfer(bob, 100);

assertEq(pyusdx.balanceOf(bob), uint256(type(uint112).max) + 49);
assertEq(pyusdx.earningPrincipalOf(bob), 49);

vm.warp(block.timestamp + 365 days);
assertEq(pyusdx.accruedYieldOf(bob), 0);
}
```

Impact

A normal inbound transfer can wrap `earningPrincipal` from near `type(uint112).max` to a dust-sized value, effectively erasing almost the entire earning base of the recipient account.

Recommendation

Remove the unchecked clause.

Issue L-10: Frozen payout addresses can block claims and account updates [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/44>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

`claimFor()` function claims accrued yield on the earner, then pays the net yield to `claimRecipient` and the fee to `earnerManager` through the generic `_transfer()` path. `_transfer()` rejects frozen recipients.

```
function _claimFor(...) private returns (...) {
    ...
    if (yieldAmount > 0) _transfer(account, claimRecipient, yieldAmount);
    if (feeAmount > 0) _transfer(account, earnerManager(), feeAmount);
}

function _transfer(...) internal override {
    ...
    _revertIfFrozen(recipient);
}
```

`_setAccountInfo()` also calls `_claimFor()` before updating the account's configuration. As a result, if the current `claimRecipient` is frozen, or if the current fee recipient is frozen while `feeRate > 0`, then ordinary `claimFor()` reverts and ordinary unpaused `setAccountInfo()` reverts before it can clear the bad configuration.

```
function _setAccountInfo(address account, uint32 earnerRate, uint16 feeRate,
    ↪ address claimRecipient) internal {
    ...
    _claimFor(account, paused());
    ...
}
```

Emergency workarounds exist, but the normal path is still blocked by an unrelated frozen payout endpoint.

Impact

Consider the case where a user's `claimRecipient` is frozen, then admin cannot modify user's `claimRecipient` by `setAccountInfo()` because it would revert on `_claimFor`. Also, if fee recipient is frozen, all `claimFor` would fail.

Recommendation

Inside `_claimFor()`, consider skipping yield and yield fee transfer if the `claimRecipient` or fee recipient is frozen.

Discussion

PierrickGT

Acknowledged.

If the Earner Manager is frozen, the admin can call `setEarnerManager(address earnerManager_)` and change of Earner Manager to resume claims. If a `claimRecipient` is frozen, yield should not be transferred to the recipient. It does indeed block the Earner Manager from changing the `claimRecipient`. In this case, two solutions are possible to resume claims.

1. Temporarily unfreeze the `claimRecipient` and call `setAccountInfo()` to change of `claimRecipient`. Not ideal since if not done atomically, the `claimRecipient` could transfer the funds before being frozen again.
2. Freeze the account with a frozen `claimRecipient` and use `forceTransfer()` to move the frozen funds and reconcile the balance sheet off-chain. Obviously, account could transfer their funds before being frozen too.

Ideally, this kind of situation should not occur and the Freeze Manager should freeze the account as well as his `claimRecipient` at the same time to avoid it.

Issue L-11: Large balances can be silently truncated during principal conversion [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/45>

Vulnerability Detail

PYUSDx converts present-value amounts into principal through `_getPrincipalAmountRoundedDown()` and `_getPrincipalAmountRoundedUp()`. Both helpers first cast the input amount from `uint256` to `uint240` before calling the indexing math library.

```
function _getPrincipalAmountRoundedDown(uint256 presentAmount, uint128 index)
↳ internal pure returns (uint112) {
    return IndexingMath.getPrincipalAmountRoundedDown(uint240(presentAmount),
↳ index);
}

function _getPrincipalAmountRoundedUp(uint256 presentAmount, uint128 index)
↳ internal pure returns (uint112) {
    return IndexingMath.getPrincipalAmountRoundedUp(uint240(presentAmount), index);
}
```

This cast is unchecked. If `presentAmount > type(uint240).max`, it is truncated modulo 2240 instead of reverting.

This matters because token balances and amount are stored as `uint256` throughout PYUSDx, and no storage uses `uint245`. Though it is very unlikely to reach `uint240` token amount, it is still recommended to remove this silent casting.

Impact

Very large balances or transfer amounts can silently produce incorrect principal, permanently understating future yield and breaking accounting instead of reverting cleanly.

Recommendation

Remove the `uint240` casting.

Issue L-12: Redundant freeze check in `_transfer` when `msg.sender == sender` [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/46>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_transfer` calls `_revertIfFrozen` on both `msg.sender` and `sender`. In the path with `transfer` or `claimFor` -> `_transfer` these are the same address.

Vulnerability Detail

The function checks both addresses to be not frozen:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDx/src/PYUSDx.sol#L558-561>

```
function _transfer(address sender, address recipient, uint256 amount) internal  
    ↪ override whenNotPaused {  
        _revertIfZeroAccount(recipient);  
        _revertIfFrozen(msg.sender);  
        _revertIfFrozen(sender);  
        _revertIfFrozen(recipient);  
    }
```

This creates a double-check situation in the case when `msg.sender == sender`.

Impact

Gas waste due to the redundant SLOAD.

Tool Used

Manual Review

Recommendation

Consider adding the following check:

```
_revertIfFrozen(sender);  
+if (msg.sender != sender) _revertIfFrozen(msg.sender);  
_revertIfFrozen(recipient);
```

Discussion

PierrickGT

Acknowledged. The micro gas optimization is not worth adding a new code path. For this reason, we won't implement the suggested recommendation.

Issue L-13: earnerRate declared as uint32 though bounded by ONE_HUNDRED_PERCENT = 10_000 [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/47>

Summary

The Account struct stores earnerRate as uint32, even though the value is bounded to ONE_HUNDRED_PERCENT = 10_000 - which fits in uint16. feeRate in the same struct is correctly typed uint16.

Vulnerability Detail

The earnerRate parameter is stored as uint32:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/PYUSD.sol#L36>

```
uint32 earnerRate;
```

However, the value can't be larger than 10_000:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSD/src/PYUSD.sol#L358>

```
if (earnerRate > ONE_HUNDRED_PERCENT) revert EarnerRateTooHigh(earnerRate);
```

Impact

No serious impact present - this only creates inconsistency between the rate variables and the parameter is left with a much wider representable range than it should be in reality.

Tool Used

Manual Review

Recommendation

Consider changing earnerRate to uint16 to align with the feeRate:

```
- uint32 earnerRate;  
+ uint16 earnerRate;
```

Issue L-14: PYUSDx freeze bypass via Portal.sendToken with extension source token [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/48>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Portal.sendToken does not check whether msg.sender is frozen on PYUSDx when the sourceToken parameter is a PYUSDx-backed extension (YieldToOne / MultiMint). Because extensions maintain their own, independent freeze lists, a user frozen on PYUSDx but unfrozen on an extension can route value off-chain via Portal: the extension is unwrapped to PYUSDx inside Portal, PYUSDx is burned **from Portal's own balance** (not the user's), and the equivalent value is minted on the destination chain. The source-chain PYUSDx freeze is never enforced against the originating user.

Vulnerability Detail

PYUSDx._transfer enforces freeze on msg.sender / sender / recipient, and PYUSDx.burn enforces freeze on the burn target account. Direct Portal.sendToken with sourceToken = pyusdx is correctly blocked because IERC20(pyusdx).safeTransferFrom(user, Portal, amount) reverts on a frozen user.

However, when sourceToken is an extension, the flow becomes:

```
function _transferAndUnwrap(address sourceToken, uint256 specifiedAmount) internal {
    ...
    // Transfer source token from the sender
    IERC20(sourceToken).safeTransferFrom(msg.sender, address(this),
    ↪ specifiedAmount);
    ...
    if (sourceToken != pyusdx) {
        ...
        IERC20(sourceToken).forceApprove(swapFacility, actualAmount);
        ISwapFacility(swapFacility).swapOut(sourceToken, actualAmount,
        ↪ address(this));
    }
    ...
}
```

and then in _sendToken:

```
// Burn PYUSDx tokens
IPYUSDx(pyusdx).burn(address(this), amount);
```


Every freeze check along this path targets Portal or SwapFacility – not the originating user:

Step	Freeze check performed on	Us
<code>Extension.transferFrom(user → Portal)</code>	user (on extension freeze list)	O
<code>SwapFacility.swapOut → Extension.unwrap</code>	<code>ISwapFacility.msgSender() = Portal</code> (on extension)	O
<code>PYUSDX.transfer(SwapFacility → Portal)</code>	SwapFacility, Portal (on PYUSDX)	Us
<code>PYUSDX.burn(Portal, amount)</code>	Portal (on PYUSDX)	Us

At no point is `_revertIfFrozen(user)` called on the **PYUSDX** freeze list. A user who is frozen on PYUSDX but not on the extension can:

1. Have held extension tokens from before being frozen (e.g., MultiMint, YieldToOne).
2. Approve Portal for their extension balance.
3. Call `Portal.sendToken(amount, extension, destChain, PYUSDX, recipient, refund Address)`.
4. Portal unwraps extension → PYUSDX into its own balance, burns PYUSDX from itself, sends cross-chain message.
5. On the destination chain, Portal mints fresh PYUSDX to the user-specified recipient (if that recipient is not frozen on destination PYUSDX).

Result: the sanctioned/frozen user successfully moves value off the chain where they were frozen, despite the PYUSDX freeze. This bypasses the compliance invariant the spec states for PYUSDX:

Frozen accounts cannot transfer, receive, or claim yield.

Note that the user cannot achieve the same bypass by unwrapping the extension directly on source (e.g., `SwapFacility.swapOut(extension, amount, user)` reverts because the final `PYUSDX.transfer(recipient=user, ...)` hits the PYUSDX freeze on the user). The bypass is **specific to Portal** because Portal intercepts the PYUSDX at itself (unfrozen) and never transfers PYUSDX to the frozen user on source.

Proof of Vulnerability

Assume: Bob frozen on PYUSDX (e.g., via `FREEZE_MANAGER_ROLE`); Bob not frozen on MultiMint; Bob holds 100 MultiMint tokens; Portal is granted `ISSUER_ROLE` on PYUSDX at deployment.

```
1. Bob.approve(Portal, 100) on MultiMint. // OK
   ↳ (_beforeApprove checks MultiMint freeze only)
2. Bob.Portal.sendToken(100, MultiMint, chainB, PYUSDX, Bob_alt, Bob_alt)
```

```

a. _transferAndUnwrap(MultiMint, 100):
  - MultiMint.transferFrom(Bob, Portal, 100)           // OK (Bob not
    ↳ frozen on MultiMint)
  - forceApprove(SwapFacility, 100)
  - SwapFacility.swapOut(MultiMint, 100, Portal):
    - MultiMint.transferFrom(Portal, SwapFacility, 100) // OK (Portal not
      ↳ frozen)
    - MultiMint.unwrap(100)                             // OK (checks
      ↳ Portal, not Bob)
    - PYUSD.transfer(SwapFacility → Portal, 100)         // OK (Portal not
      ↳ frozen on PYUSD)
b. PYUSD.burn(Portal, 100)                             // OK (Portal not
  ↳ frozen on PYUSD)
c. adapter.sendMessage(...)                             // OK
3. On chainB: Portal.receiveMessage → PYUSD.mint(Bob_alt, 100) // OK (Bob_alt not
  ↳ frozen on chainB PYUSD)

```

Bob moves 100 tokens off chain despite being frozen on source PYUSD. No step reverts.

Impact

Users frozen on PYUSD retain a live pathway to move value off the chain, defeating regulatory/sanctions freezes.

Code Snippet

```

// portal/Portal.sol:394-438 (_sendToken, relevant portion)
function _sendToken(
    uint256 amount,
    address sourceToken,
    uint32 destinationChainId,
    bytes32 destinationToken,
    bytes32 recipient,
    bytes32 refundAddress,
    address bridgeAdapter
) internal returns (bytes32 messageId) {
    _revertIfZeroAmount(amount);
    _revertIfZeroRefundAddress(refundAddress);
    _revertIfZeroSourceToken(sourceToken);
    _revertIfZeroDestinationToken(destinationToken);
    _revertIfZeroRecipient(recipient);
    _revertIfUnsupportedBridgeAdapter(destinationChainId, bridgeAdapter);

    // NOTE: no _revertIfFrozen(msg.sender) on PYUSD

    _transferAndUnwrap(sourceToken, amount);

```

```

// Burn PYUSDx tokens
IPYUSDx(pyusdx).burn(address(this), amount);
...
}

```

```

// portal/Portal.sol:445-472 (_transferAndUnwrap)
function _transferAndUnwrap(address sourceToken, uint256 specifiedAmount) internal {
    uint256 balanceBefore = _pyusdxBalanceOf(address(this));
    uint256 sourceTokenBalanceBefore = _tokenBalanceOf(sourceToken, address(this));

    // Transfer source token from the sender - freeze-checked on SOURCE TOKEN only
    IERC20(sourceToken).safeTransferFrom(msg.sender, address(this),
        ↪ specifiedAmount);

    if (sourceToken != pyusdx) {
        ...
        // SwapFacility.swapOut → Extension.unwrap → PYUSDx.transfer(SwapFacility →
        ↪ Portal)
        // None of these checks the USER's PYUSDx freeze status.
        IERC20(sourceToken).forceApprove(swapFacility, actualAmount);
        ISwapFacility(swapFacility).swapOut(sourceToken, actualAmount,
            ↪ address(this));
    }
    ...
}

```

Tool Used

Manual Review

Recommendation

Add an explicit PYUSDx-freeze check on `msg.sender` at the entry point of `sendToken` (both overloads), independent of the source token:

```

function sendToken(
    uint256 amount,
    address sourceToken,
    uint32 destinationChainId,
    bytes32 destinationToken,
    bytes32 recipient,
    bytes32 refundAddress
) external payable whenSendNotPaused whenNotLocked returns (bytes32 messageId) {
+   if (IFreezable(pyusdx).isFrozen(msg.sender)) revert SenderFrozen(msg.sender);
    return
        _sendToken(
            amount,

```

```
        sourceToken,  
        destinationChainId,  
        destinationToken,  
        recipient,  
        refundAddress,  
        defaultBridgeAdapter(destinationChainId)  
    );  
}
```

Apply the same check to the overloaded `sendToken(..., address bridgeAdapter)`. Alternatively, enforce this inside `_sendToken` so both overloads are covered in one place.

Discussion

PierrickGT

Acknowledged. Ideally the entity in charge of the Freeze Manager role should freeze an account on both, the extension and PYUSD, as well as across all supported chains. It is also not the role of the Portal to check the frozen status of accounts. For these reasons, we won't implement the suggested change.

Issue L-15: quote ignores sendPaused, returning stale estimates during pause [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/50>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

quote does not revert when sending is paused, so off-chain callers and integrators receive a fee estimate for an operation that will immediately fail.

Vulnerability Detail

sendToken is gated by the whenSendNotPaused modifier and reverts with SendingPaused while sendPaused is true. The view helpers quote and _quote do not check sendPaused and keep returning a valid fee quote from the bridge adapter even when outbound messaging is paused.

Integrators and frontends typically call quote to pre-fund msg.value before invoking sendToken. During a pause window they get a normal quote with no signal that the send path is disabled, wasting gas on doomed transactions and masking an operationally critical state.

Impact

Misleading off-chain UX during an incident pause. Users and integrators cannot distinguish between a healthy route and one that is paused, since quote returns as if sending were enabled.

Code Snippet

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDX/src/portal/Portal.sol#L357-L364>

```
function quote(uint32 destinationChainId) external view returns (uint256) {
    return _quote(destinationChainId, defaultBridgeAdapter(destinationChainId));
}

function quote(uint32 destinationChainId, address bridgeAdapter) external view
↪ returns (uint256) {
    return _quote(destinationChainId, bridgeAdapter);
}
```

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/blob/main/PYUSDx/src/portal/Portal.sol#L577-L588>

```
function _quote(uint32 destinationChainId, address bridgeAdapter) internal view
↳ returns (uint256) {
    _revertIfUnsupportedBridgeAdapter(destinationChainId, bridgeAdapter);

    uint256 gasLimit = _getPayloadGasLimitOrRevert(destinationChainId);

    bytes memory payload = PayloadEncoder.generateEmptyPayload();

    return IBridgeAdapter(bridgeAdapter).quote(destinationChainId, gasLimit,
↳ payload);
}
```

Tool Used

Manual Review

Recommendation

Apply the same pause gate used by `sendToken` to the `quote` path so callers get a clear signal when sending is disabled.

```
function _quote(uint32 destinationChainId, address bridgeAdapter) internal view
↳ returns (uint256) {
    if (sendPaused()) revert SendingPaused();
    _revertIfUnsupportedBridgeAdapter(destinationChainId, bridgeAdapter);
    ...
}
```

Discussion

PierrickGT

Acknowledged.

This is a view function, so ideally it should not revert and regardless of whether the contract is paused, the quoting mechanism of the `bridgeAdapter` should still be working. An integrator should first call `sendPaused()` if they don't want to waste gas on a failing `sendToken` call.

For these reasons, we won't implement the recommendation.

Issue L-16: setFallbackRecipient does not validate that the new recipient is currently unfrozen on PYUSDx [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/51>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`Portal.setFallbackRecipient` only rejects the zero address; it does not check whether the candidate fallback is currently frozen on PYUSDx. The PYUSDx-portal doc explicitly states the fallback recipient is "expected to remain unfrozen", but the contract does not enforce this invariant at the moment of assignment. If the admin mistakenly (or a compromised admin deliberately) sets the fallback to an address that is already on the PYUSDx freeze list, every subsequent cross-chain message destined for a frozen recipient will hit the redirect-to-fallback path and revert at `PYUSDx.mint(fallback, amount)`, leaving messages permanently stuck until admin rotates the fallback.

Vulnerability Detail

```
function setFallbackRecipient(address fallbackRecipient_) external
↳ onlyRole(DEFAULT_ADMIN_ROLE) {
    _setFallbackRecipient(fallbackRecipient_);
}

function _setFallbackRecipient(address fallbackRecipient_) internal {
    if (fallbackRecipient_ == address(0)) revert ZeroFallbackRecipient();

    PortalStorageStruct storage $ = _getPortalStorageLocation();
    if ($.fallbackRecipient == fallbackRecipient_) return;

    $.fallbackRecipient = fallbackRecipient_;
    emit FallbackRecipientSet(fallbackRecipient_);
}
```

No `IFreezable(pyusdx).isFrozen(fallbackRecipient_)` check. If the set fallback is or becomes frozen, the spec-documented "retryable after admin rotates fallbackRecipient" condition becomes the primary failure mode.

Impact

Subtle misconfiguration by a well-intentioned admin: typo / wrong address set as fallback that happens to be already frozen. All frozen-recipient-redirected messages revert until admin notices and rotates.

Code Snippet

```
// portal/Portal.sol:271-275
function setFallbackRecipient(address fallbackRecipient_) external
↳ onlyRole(DEFAULT_ADMIN_ROLE) {
    _setFallbackRecipient(fallbackRecipient_);
}

// portal/Portal.sol:556-564
function _setFallbackRecipient(address fallbackRecipient_) internal {
    if (fallbackRecipient_ == address(0)) revert ZeroFallbackRecipient();

    PortalStorageStruct storage $ = _getPortalStorageLocation();
    if ($.fallbackRecipient == fallbackRecipient_) return;

    $.fallbackRecipient = fallbackRecipient_;
    emit FallbackRecipientSet(fallbackRecipient_);
}
```

Tool Used

Manual Review

Recommendation

Enforce the unfrozen invariant at assignment:

```
function _setFallbackRecipient(address fallbackRecipient_) internal {
    if (fallbackRecipient_ == address(0)) revert ZeroFallbackRecipient();
+   if (IFreezable(pyusdx).isFrozen(fallbackRecipient_)) revert
↳ FallbackRecipientFrozen(fallbackRecipient_);

    PortalStorageStruct storage $ = _getPortalStorageLocation();
    if ($.fallbackRecipient == fallbackRecipient_) return;

    $.fallbackRecipient = fallbackRecipient_;
    emit FallbackRecipientSet(fallbackRecipient_);
}
```


Discussion

PierrickGT

Acknowledged.

It's not the role of the Portal to check the frozen status of the passed `fallbackRecipient_`. The Admin should ensure that they are setting a `fallbackRecipient` that is not frozen.

For these reasons, we won't implement the recommended change.

Issue L-17: Revoked Operator Can Still Cancel Their Own Pending Mint Proposals [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-04-m0-pyusdx-apr-20th-2026/issues/52>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability detail

In `IssuerGateway.sol`, `cancelMint()` authenticates the caller by proposal ownership, not by current role membership. The function requires `proposal.minter == msg.sender`, but it does not require the caller to still hold `OPERATOR_ROLE`.

As a result, an operator who created a mint proposal and was later offboarded can still cancel that proposal as long as it remains pending. Role revocation therefore does not fully remove that operator's influence over pre-existing proposals.

This may be intentional from a UX perspective, but it weakens the security/operations assumption that revoking `OPERATOR_ROLE` fully cuts off the actor from the mint workflow.

```
function cancelMint(uint48 mintId) external {
    IssuerGatewayStorage storage $ = _getIssuerGatewayStorage();
    MintProposal storage proposal = $.mintProposals[mintId];

    if (proposal.createdAt == 0) revert InvalidMintProposal();
    @> if (proposal.minter != msg.sender) revert NotMintProposalCreator();

    uint40 activeAt;

    // NOTE: safe to use unchecked, uint40 timestamps overflow past year 36812,
    //         and max uint32 delay (~136 years) cannot push them past that until
    //         then.
    unchecked {
        activeAt = proposal.createdAt + $.mintDelay;
    }

    if (block.timestamp >= activeAt) revert ActiveMintProposal(activeAt);

    delete $.mintProposals[mintId];

    emit MintCanceled(mintId, msg.sender);
}
```

Impact

A revoked `OPERATOR_ROLE` cannot create new proposals, but can still interfere with previously created pending proposals by canceling them unilaterally. This creates inconsistent/incomplete offboarding and may disrupt operational mint flows if governance or admins assume role revocation fully neutralizes the former operator.

Recommendation

Require both conditions inside `cancelMint()`:

- the caller is the original proposal creator
- the caller still holds `OPERATOR_ROLE`

Alternatively, provide an explicit admin-controlled cancellation path and treat revoked operators as fully deauthorized from interacting with outstanding proposals.

Discussion

PierrickGT

Acknowledged.

On the flip side, if we add the `onlyRole(OPERATOR_ROLE)` check, a revoked Operator wouldn't be able to cancel their pending mint proposal and any address with the `EXECUTOR_ROLE` could mint it once the proposal active. For this reason, we won't implement the recommendation.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.